

Adapting Action Selection for Frozen VLAs from Deployment Experience

Doyoung Kwon¹, Hyegang Kim², Chenguang Li³, Wooju Kim^{1,*}, and Chi-Guhn Lee^{2,*}

Abstract—Vision-Language-Action (VLA) models may underperform when deployment conditions differ from those represented in training, while collecting sufficient target-domain data for policy adaptation can be costly. We study how to adapt the behavior of a frozen VLA using a small set of successful target-task demonstrations and repeated deployment experience. Our framework augments native VLA proposals with retrieved and memory-initialized candidates derived from demonstration memory, and uses a lightweight external selector to choose among them. The selector is initialized from short-horizon physical preferences on demonstration states. After each deployment round, candidates are counterfactually evaluated from states visited by the current selector, producing new pairwise preferences that update only the selector while leaving the VLA fixed. We evaluate the framework using a LIBERO-finetuned $\pi_{0.5}$ on four ManiSkill3 manipulation tasks. Our method reaches deeper task stages than the frozen VLA and a retrieval-based in-context baseline in initial deployment. Across repeated deployment rounds, selector adaptation yields overall improvements in later-stage progress, including in settings with no full-task success. Over 98% of deployment-visited states also yield informative candidate preferences. These results provide initial evidence that few demonstrations and deployment experience can jointly specialize action selection under distribution shift without updating the VLA.

I. INTRODUCTION

Vision-Language-Action (VLA) models have emerged as generalist robot policies with broad manipulation capabilities across diverse tasks and environments [1], [2], [3], [4]. In industrial deployment, however, a general-purpose policy must be specialized for a specific production condition, where task configuration, observations, and interaction dynamics may differ from those represented in training [5], [6], [7], [8]. A common approach to bridging this gap is to fine-tune the pretrained VLA on demonstrations collected from the target domain [3], [9], [10]. However, as deployment exposes states beyond the initial adaptation data, continued specialization through repeated VLA updates may risk degrading capabilities retained from pretraining [11]. This motivates complementary approaches that adapt the behavior of a frozen VLA [12], [13], [14], [15].

*Corresponding authors.

¹Doyoung Kwon and Wooju Kim are with the Department of Industrial Engineering, Yonsei University, Seoul, Republic of Korea. dykwon0312@yonsei.ac.kr, wkim@yonsei.ac.kr

²Hyegang Kim and Chi-Guhn Lee are with the Department of Mechanical & Industrial Engineering, University of Toronto, Toronto, ON, Canada. hyegang.kim@mail.utoronto.ca, cglee@mie.utoronto.ca

³Chenguang Li is with the Department of Electrical and Computer Engineering, University of Toronto, Toronto, ON, Canada. chenguang.li@mail.utoronto.ca

We ask whether such adaptation can be achieved from limited target-task data. A small set of successful demonstrations provides task-specific information that can be reused at deployment, and retrieval-based in-context methods offer one way to do so without further parameter updates [16]. However, relying on demonstration reuse alone has two limitations. First, retrieved actions originate from demonstration states and may not be directly appropriate for the robot’s current state [17], [18], [19]. Second, closed-loop execution can drive the policy into states not covered by a small demonstration memory [20]. At these deployment-visited states, sparse terminal outcomes provide little supervision for distinguishing actions that make meaningful intermediate progress [21].

To address these challenges, we introduce a deployment-state adaptation framework that keeps the VLA frozen and updates an external action selector. At each replanning step, the selector chooses from a candidate pool comprising native VLA proposals, raw retrieved actions, and memory-initialized proposals generated from the current observation. We initialize the selector on demonstration-memory states using pairwise preferences derived from short-horizon physical progress. After each deployment round, candidates are counterfactually executed from saved deployment states for short-horizon evaluation, and the resulting preferences are used to update the selector for the next round. Because this supervision relies on relative progress rather than terminal success alone, adaptation can continue even when a deployment round contains no successful trajectory. Thus, few demonstrations initialize task-relevant action selection, while repeated deployment experience specializes the selector to the states it actually encounters.

We evaluate this setting by deploying a frozen LIBERO-finetuned $\pi_{0.5}$ on four ManiSkill3 manipulation tasks [22], [23]. In initial deployment, our method reaches deeper task stages than the frozen $\pi_{0.5}$ model and a π_0 -FAST retrieval-based in-context baseline across all four tasks, with higher full-task success on PullCube and PushCube. On PullCube, task success changes from 50.0% in Round 1 to 45.0% in Round 2 and 77.5% in Round 3. On PegInsertionSide, no round achieves full-task success, yet later rounds demonstrate more progress beyond contact toward grasping and lifting the peg. Across both tasks, over 98% of deployment-visited states provide at least one strict candidate preference, showing that adaptation supervision remains widely available even in rounds with no successful trajectory.

Our contributions are twofold:

- We construct a task-relevant candidate pool for frozen-

VLA deployment adaptation using a small target-task demonstration memory, combining native proposals with retrieved and memory-initialized actions.

- We introduce round-wise adaptation of an external action selector using pairwise preferences derived from counterfactual short-horizon progress at deployment-visited states, enabling adaptation even when terminal task success is absent.

II. RELATED WORK

A. Deployment-Time Adaptation and Action Selection for Frozen VLAs

Recent work has explored adapting robot policies from deployment interaction. TT-VLA and EVOLVE-VLA update the action policy using environment feedback [24], [25], whereas TTT-VLA keeps the policy backbone frozen and adapts a latent interface at test time that is learned jointly with the policy beforehand [26]. Failure-oriented methods pursue related goals through test-time policy adaptation, as in FAR, or auxiliary residual recovery policies while keeping the VLA frozen, as in ReCoVLA [27], [28]. Our approach instead leaves the VLA action generator unchanged and places deployment-time adaptation in an external action selector, without requiring an adaptation-specific interface within the VLA.

A complementary line of work keeps the base policy frozen and improves execution by externally evaluating and selecting among its action candidates. SVA selects among frozen-VLA candidates using a learned value model [15], UF-OPS learns a verifier from initial on-policy rollouts [14], and RoboCritic learns fine-grained progress comparisons from successful and failed rollouts [29]. In these methods, the evaluator is trained before the reported test-time selection phase, rather than being repeatedly retrained on states induced by the current selector during deployment. Other external evaluators are similarly used for test-time action selection [30], [31]. Most closely related, B2FF trains a selector from counterfactual candidate rollouts at injected failure contexts using binary recovery outcomes, and then keeps the selector fixed during deployment [32]. In contrast, we repeatedly update the external action selector from states visited under its current deployment policy, using short-horizon relative physical progress to supervise the next-round selector.

B. Progress- and Preference-Based Supervision

Prior work has shown that richer progress signals can improve learning beyond sparse terminal success. ReWiND and RARM construct dense reward signals using learned reward models or reference-anchored progress [33], [21]. ARM models relative advantage from pairwise state comparisons [34], while Robometer combines frame-level progress supervision with inter-trajectory preferences [35]. ALTER derives stage-relative advantage labels from local progress within a fixed deployment corpus, without additional environment interaction, for offline policy learning [36]. LfH relabels failed VLA rollouts according to behaviors they

actually accomplish [37]. These methods use progress-, preference-, or hindsight-based supervision to train reward models or improve the underlying robot policy.

Our distinction lies in how progress supervision is generated and used during deployment. After each deployment round, we restore states actually visited by the current selector, execute alternative candidates from the same restored state, and convert their short-horizon physical outcomes into pairwise preferences. These preferences are used to update the external selector while keeping the VLA action generator frozen, providing supervision directly from the selector-induced deployment distribution even in rounds with no successful trajectory.

C. Retrieval-Augmented VLA Execution

Retrieval offers a lightweight way to exploit small demonstration or experience memories. RICL post-trains π_0 -FAST for retrieval-augmented in-context learning and interpolates the closest retrieved action tokens with the model’s autoregressive action-token distribution [16]. Related work uses retrieved memories either to augment action generation in frozen VLAs or to support policy adaptation [38], [39]. Retrieve-then-Steer retrieves successful action experience and injects it as a prior into the sampler of a frozen flow-matching VLA [19].

We instead use retrieval primarily to expand the action candidates available to an external selector, retaining retrieved and memory-initialized actions alongside native VLA proposals. Memory-initialized sampling is conceptually related to the prior-guided generation in Retrieve-then-Steer, but here it produces only one candidate family that is subsequently compared with native and retrieved candidates by an external selector [19].

III. METHODOLOGY

A. Deployment Setting

We consider repeated closed-loop deployment of a frozen VLA policy π in a target task that differs from its training distribution. A small demonstration memory $\mathcal{M}$ contains successful target-task trajectories collected under initial configurations different from those used for evaluation. These demonstrations are used for candidate construction and selector training.

At each replanning step t , we construct a set of N action-chunk candidates $\mathcal{C}_t = \{\mathbf{a}_{t,1}, \dots, \mathbf{a}_{t,N}\}$ using the frozen VLA and demonstration memory $\mathcal{M}$, with each candidate having horizon H . A lightweight selector f_θ scores the candidates, and the highest-scoring candidate is selected for execution. Only the first H_e actions of the selected chunk are executed before the next replanning step. Candidate construction is described in Sec. III-B and selector scoring in Sec. III-C.

Deployment states and their corresponding candidate sets are stored for the offline preference supervision described in Sec. III-D. Across deployment rounds, we update only the selector while keeping the VLA and candidate-construction procedure fixed.

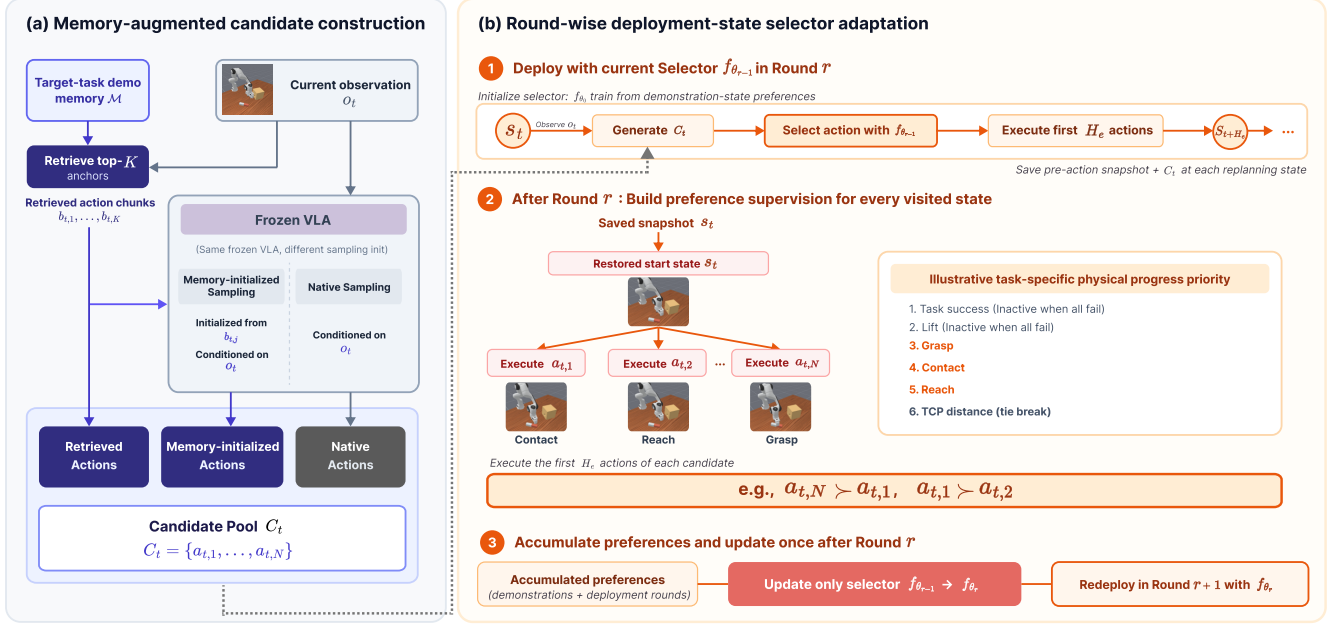


Fig. 1. Overview of our framework. (a) A small target-task demonstration memory augments native frozen-VLA proposals with retrieved actions and memory-initialized proposals. (b) During deployment, pre-action states and candidate sets are stored at each replanning step. After each round, stored states are restored and candidate prefixes are counterfactually evaluated from the same state using task-specific physical progress. Preferences aggregated across visited states update only the selector before the next deployment round; the VLA remains frozen.

B. Memory-Augmented Candidate Construction

At each replanning step t , we augment the frozen VLA’s native proposals using the demonstration memory $\mathcal{M}$. We retrieve the K most relevant demonstration anchors, where each anchor denotes the starting time step of an H -step action chunk in a demonstration trajectory. Retrieval compares temporal visual features, robot-state features, and their temporal differences after robust normalization, using only deployment-available observations and no privileged object or task state. Appendix A provides a brief analysis of the choice of K .

Each retrieved anchor $j \in \{1, \dots, K\}$ provides an action chunk $\mathbf{b}_{t,j} \in \mathbb{R}^{H \times d_a}$, where d_a denotes the action dimension. Retrieved observations are used only for retrieval; VLA sampling remains conditioned on the current deployment observation.

The candidate pool contains both native VLA proposals and actions derived from the retrieved chunks. We first generate one nominal and eight stochastic action chunks directly from the frozen VLA. We set $K = 2$ for all experiments, and the two retrieved action chunks are also included directly as raw retrieved candidates. In addition, each retrieved action chunk is used to initialize the VLA’s flow-matching sampler. After mapping the retrieved chunk to the VLA’s normalized action-chunk space, we construct

$$\mathbf{x}_{t,j,\lambda} = (1 - \lambda)\hat{\mathbf{b}}_{t,j} + \lambda\epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad (1)$$

where $\hat{\mathbf{b}}_{t,j} \in \mathbb{R}^{H \times d_a}$ denotes the normalized retrieved action chunk, ϵ is standard Gaussian noise of the same shape, and λ controls the amount of injected noise. Starting from $\mathbf{x}_{t,j,\lambda}$,

the frozen sampler continues toward the action endpoint while conditioning on the current deployment observation. We refer to the resulting proposals as *memory-initialized* candidates. We use $\lambda \in \{0.05, 0.10, 0.20\}$ and draw two independent noise samples for each retrieved anchor and initialization level, yielding twelve memory-initialized proposals. Appendix B analyzes the complementary candidate support provided by the three initialization levels.

The resulting candidate pool is

$$\mathcal{C}_t = \{\mathbf{a}_{t,1}, \dots, \mathbf{a}_{t,N}\}, \quad (2)$$

where each $\mathbf{a}_{t,i} \in \mathbb{R}^{H \times d_a}$ denotes a candidate action chunk. In our experiments, $N = 23$, comprising nine native VLA proposals, two raw retrieved chunks, and twelve memory-initialized proposals.

C. Context-Conditioned Action Selector

Given the candidate set $\mathcal{C}_t$ from Sec. III-B, a lightweight selector f_θ assigns a scalar score to each candidate. The execution context c_t contains a short history of wrist-camera DINOv2 features, proprioceptive states, and their temporal differences [40].

For each candidate $\mathbf{a}_{t,i}$, we construct a candidate representation $\mathbf{u}_{t,i}$ from its normalized action chunk and a small set of candidate-generation metadata. A lightweight two-tower MLP separately encodes the execution context c_t and candidate representation $\mathbf{u}_{t,i}$, then combines the resulting embeddings to produce a scalar score

$$z_{t,i} = f_\theta(c_t, \mathbf{u}_{t,i}). \quad (3)$$

The highest-scoring candidate is selected as

$$i_t^* = \arg \max_{i \in \{1, \dots, N\}} z_{t,i}. \quad (4)$$

D. Deployment-State Preference Supervision and Iterative Adaptation

To train the selector, we construct pairwise preferences by counterfactually evaluating candidate action chunks from a common simulator state. For initial training, we use states drawn from the successful demonstration trajectories in $\mathcal{M}$. At each such state, we construct a candidate set using the procedure in Sec. III-B and evaluate each candidate by executing its first H_e actions from the same starting state. The resulting behavior is evaluated using a task-specific lexicographic physical progress measure that prioritizes task completion and intermediate manipulation progress. This allows preferences to be formed from partial physical progress even when neither candidate completes the task. Counterfactual branch evaluation and privileged physical quantities are used only offline to generate selector supervision and are never used for online selection.

At a given training state, we suppress the state index for clarity. For two distinct candidate indices $p, q \in \{1, \dots, N\}$, if candidate $\mathbf{a}_p$ achieves strictly greater physical progress than $\mathbf{a}_q$, we assign the preference

$$\mathbf{a}_p \succ \mathbf{a}_q. \quad (5)$$

Ties are excluded from training. Because the selector only needs to rank candidates within the same state, we learn from relative preferences rather than regressing absolute physical scores:

$$\mathcal{L}_{\text{rank}} = \text{softplus}(-[z_p - z_q]). \quad (6)$$

Preferences constructed from demonstration states are used to train the initial selector f_{θ_0} .

We then adapt the selector using states encountered during closed-loop deployment. Let s_t denote the pre-action simulator state at replanning step t . In deployment round $r \geq 1$, we execute the current selector $f_{\theta_{r-1}}$ and, at each replanning step, store s_t together with the candidate set $\mathcal{C}_t$ generated at that state. Candidate construction continues to use the fixed demonstration memory $\mathcal{M}$ as described in Sec. III-B.

After each deployment round, the stored states are relabeled offline. For each candidate $\mathbf{a}_{t,i} \in \mathcal{C}_t$, we restore the same stored state s_t and execute the first H_e actions of that candidate. The resulting short-horizon physical outcomes are compared using the same progress measure to produce new pairwise preferences at states actually visited under the current selector.

The newly collected deployment-state preferences are accumulated with the initial demonstration-based supervision and preferences from previous deployment rounds. To obtain the next selector, we initialize θ_r from θ_{r-1} and update only the selector parameters. The VLA, demonstration memory, and candidate-construction procedure remain fixed. Repeating this procedure yields

$$\theta_0 \rightarrow \theta_1 \rightarrow \theta_2 \rightarrow \dots, \quad (7)$$

allowing the selector to specialize to the closed-loop state distribution induced by its own deployment behavior without modifying the VLA.

IV. EXPERIMENTS

A. Experimental Setup

We evaluate our method on four ManiSkill3 manipulation tasks : PlugCharger-v1, PegInsertionSide-v1, PullCube-v1, and PushCube-v1. Our method uses a LIBERO-finetuned $\pi_{0.5}$ as the VLA backbone, which remains frozen during deployment and selector adaptation [23], [22]. For each task, we collect eight successful target-task demonstrations under initial configurations separate from those used for evaluation. These demonstrations form the memory $\mathcal{M}$ used for candidate construction and for training the initial selector f_{θ_0} .

By default, our method uses the combined pool of native and memory-derived candidates described in Sec. III-B, with $N = 23$ action chunks at each replanning step. Each candidate has horizon $H = 10$, and only the first $H_e = 3$ actions of the selected chunk are executed before replanning. Round r uses selector $f_{\theta_{r-1}}$; between rounds, only the selector is updated, while the VLA and candidate-construction procedure remain fixed. Candidate-source ablations compare the combined pool against native-only and memory-derived-only variants; see Appendix C.

For each task, we evaluate four deployment seeds with 10 stochastic closed-loop trials per seed, yielding 40 trials for each evaluated method or deployment round. We report full-task success together with task-specific intermediate physical progress, including contact, grasp, lift, and goal-directed object motion where applicable. These intermediate stages quantify episode-level behavioral progress even when full-task success is sparse. The finer-grained lexicographic criteria used to generate selector preferences are distinct from these evaluation stages and are provided in Appendix D.

We evaluate initial deployment on all four tasks against the frozen LIBERO-finetuned $\pi_{0.5}$ model alone and a π_0 -FAST retrieval-based in-context learning (RICL) baseline [16], [22]. We use π_0 -FAST for RICL following its published formulation, as its action-interpolation mechanism is not directly applicable to the flow-matching action head of $\pi_{0.5}$. We post-train the RICL model on 20 LIBERO tasks using 20 demonstrations per task. The same eight target-task demonstrations used by our method are provided to RICL as its retrieval memory.

We then study round-wise selector adaptation over three deployment rounds on PullCube and PegInsertionSide. In initial deployment, PullCube and PushCube achieve full-task success in some trials, whereas PegInsertionSide and PlugCharger do not. We select one task from each regime—PullCube and PegInsertionSide—to examine adaptation under contrasting levels of initial task completion.

B. Initial Deployment under Domain Shift

We first evaluate initial deployment performance before incorporating any deployment-state adaptation. We compare

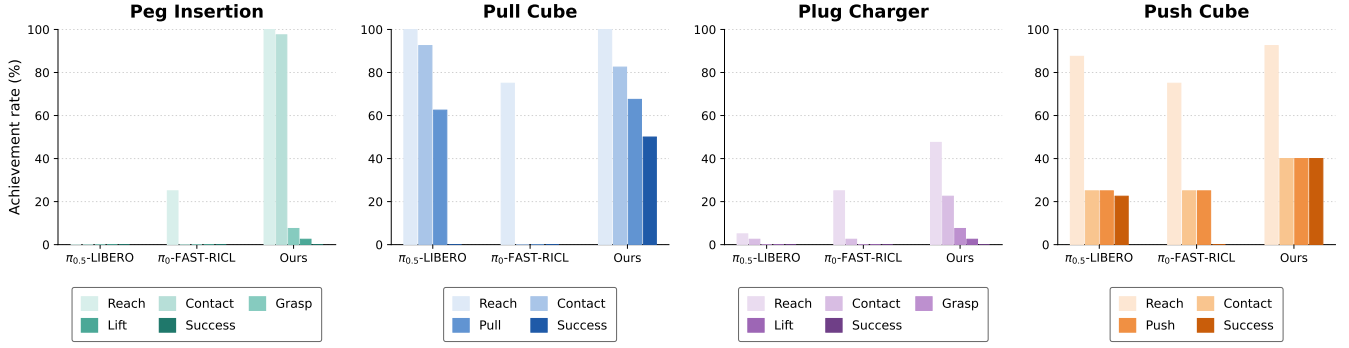


Fig. 2. Initial deployment under domain shift across four ManiSkill3 tasks. We compare the frozen LIBERO-finetuned $\pi_{0.5}$ model, the π_0 -FAST RICL baseline, and our method using the same frozen $\pi_{0.5}$ backbone with the initial selector f_{θ_0} .

the frozen LIBERO-finetuned $\pi_{0.5}$ model, the π_0 -FAST retrieval-based in-context learning (RICL) baseline [22], [16], and our method using the initial selector f_{θ_0} across the four ManiSkill3 tasks. At this stage, f_{θ_0} is trained only from demonstration-based supervision and has not yet been adapted using states encountered during deployment.

Fig. 2 compares full-task success and task-specific intermediate physical progress. The frozen $\pi_{0.5}$ model and RICL baseline show limited later-stage progress under the shifted deployment conditions. Similar sensitivity to distribution shift has been reported for $\pi_{0.5}$, with the pretrained model achieving 0% success on an unseen LIBERO suite before target-specific adaptation [41]. In contrast, our selector-augmented frozen $\pi_{0.5}$ model reaches later task stages more consistently. On PullCube and PushCube, this results in full-task success in a subset of trials, while PegInsertionSide and PlugCharger remain incomplete but exhibit intermediate manipulation progress. Stage-wise evaluation also captures partial task progress that terminal success alone does not reveal. For example, on PullCube, the $\pi_{0.5}$ model frequently satisfies the *Pull* stage by moving the cube at least 2 cm, but does not achieve full-task success.

A candidate-source ablation further shows that including memory-derived candidates provides substantial gains over native-only sampling in initial deployment, particularly for task-relevant intermediate progress and full-task success on PullCube and PushCube; see Appendix C. The relative performance of the Combined and Memory-derived only pools varies across tasks, so we do not claim that a single candidate-pool composition is uniformly optimal. Overall, these results indicate that augmenting native VLA proposals with task-relevant candidates from a small demonstration memory improves initial frozen-VLA deployment before any deployment-state adaptation.

C. Iterative Deployment-State Adaptation

We next evaluate whether supervision from deployment-visited states can further specialize the selector over repeated deployment rounds. We study PegInsertionSide and PullCube over three rounds, with four deployment seeds and 10 stochastic trials per seed (40 trials per round). Round 1 uses f_{θ_0} , while Rounds 2 and 3 use f_{θ_1} and f_{θ_2} , respectively.

After each of the first two rounds, counterfactual physical preferences collected at states visited in that round are accumulated with previous supervision to train the selector for the next round. Fig. 3 reports the fraction of trials reaching successive task-specific manipulation stages.

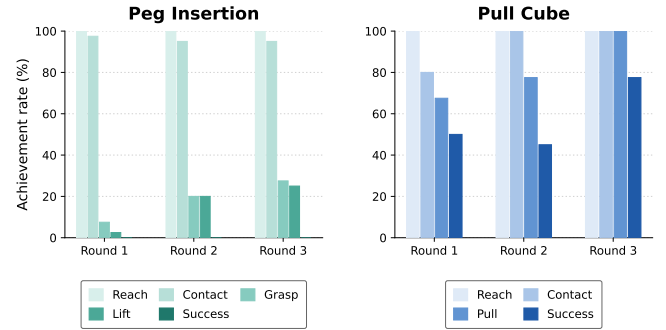


Fig. 3. Iterative deployment-state adaptation on PegInsertionSide and PullCube. Bars show the fraction of 40 closed-loop trials reaching each task-specific manipulation stage over three deployment rounds.

On PegInsertionSide, full-task success remains zero across all rounds, but later-stage progress improves substantially: Grasp increases from 7.5% in Round 1 to 27.5% in Round 3, while Lift increases from 2.5% to 25.0%. Qualitatively, Round 3 rollouts progress beyond contact and lifting to bringing the grasped peg to the insertion hole and attempting insertion. On PullCube, task success improves overall from 50.0% in Round 1 to 77.5% in Round 3, despite a slight dip in Round 2. The *Pull* stage similarly increases from 67.5% to 100%. Overall, both tasks show deeper task progress by Round 3, although individual selector updates do not always yield monotonic gains.

Importantly, informative supervision remains available even when full-task success is absent. A deployment-visited state is considered informative if counterfactual evaluation yields at least one strict preference between candidates under the short-horizon physical progress measure. Across the two relabeling rounds of PullCube and PegInsertionSide, over 98% of visited states are informative.

V. DISCUSSION AND LIMITATIONS

Our current adaptation procedure relies on restoring previously visited states and privileged task-state information (used only for offline preference supervision), limiting the present implementation to simulation or resettable environments. Physical deployment could instead pair the real robot with a digital twin for state reconstruction, counterfactual evaluation, and progress supervision. Adaptation is also not guaranteed to improve performance after every selector update, as short-horizon supervision can overemphasize locally favorable progress, motivating stability-aware update strategies. Finally, the physical-progress criteria are currently specified per task, leaving their automatic construction to future work.

VI. CONCLUSION

We presented a framework for adapting action selection around a frozen VLA using deployment experience, while leaving the underlying VLA unchanged. A small demonstration memory provides task-relevant candidates and initial selector supervision, while counterfactual short-horizon physical preferences at deployment-visited states enable round-wise selector updates. Across four ManiSkill3 tasks, our results provide initial evidence that this approach improves frozen-VLA deployment and that useful preference supervision is available for over 98% of relabeled deployment states, including settings where full-task success is absent.

APPENDIX

A. Number of Retrieved Memory Anchors

We use $K = 2$ retrieved memory anchors in all experiments. To examine this choice, we measure the diversity of the top- K retrieved action chunks over the analyzed deployment states. Action chunks are normalized per dimension and flattened, and diversity is measured as the mean pairwise ℓ_2 distance among the retrieved chunks.

As shown in Table I, increasing K from 2 to 3 or 4 provides only modest additional pairwise diversity relative to the corresponding increase in memory-conditioned sampling cost. We therefore fix $K = 2$ as a diversity–compute trade-off.

TABLE I
EFFECT OF THE NUMBER OF RETRIEVED MEMORY ANCHORS
ON ACTION DIVERSITY.

K	2	3	4
Pairwise diversity	2.077	2.265	2.412
Increase vs. $K = 2$	–	+9.0%	+16.1%

B. Memory-Initialization Levels

We evaluate the contribution of the three memory-initialization levels across four tasks. Removing $\lambda = 0.05$, 0.10, or 0.20 from the candidate pool degrades the best available physical outcome in 37.7%, 16.4%, and 32.3% of evaluated states, respectively. These results indicate that all three initialization levels provide complementary candidate support, so we retain all three for experiments.

C. Effect of Memory-Derived Candidates

We further examine the contribution of memory-derived candidates by comparing three candidate-pool compositions: *Native-only*, which contains only proposals sampled directly from the frozen VLA; *Memory-derived only*, which contains raw retrieved actions and memory-initialized proposals; and *Combined*, which contains both native and memory-derived candidates. To keep the candidate-pool size fixed at $N = 23$, we adjust the number of stochastic samples within each variant accordingly: *Native-only* uses one nominal and 22 stochastic native proposals; *Memory-derived only* uses two raw retrieved actions and 21 memory-initialized proposals; and *Combined* uses nine native, two raw retrieved, and twelve memory-initialized proposals.

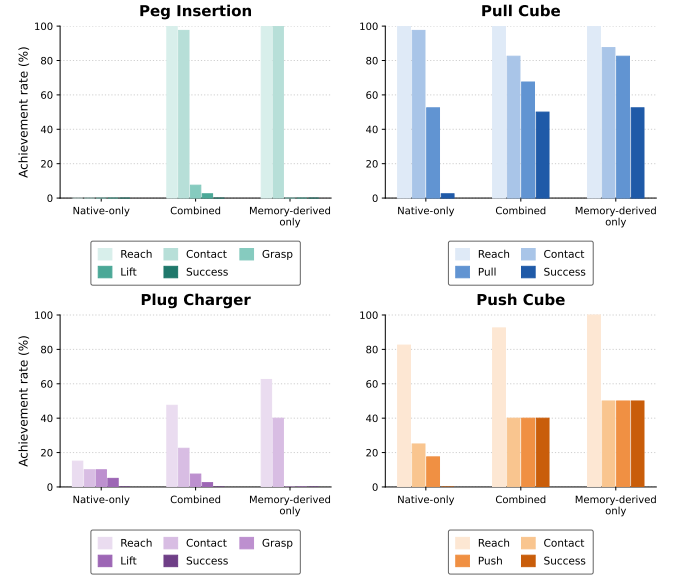


Fig. 4. Effect of candidate-pool composition on initial deployment. All variants use $N = 23$ candidates. *Native-only* uses frozen-VLA proposals only; *Memory-derived only* uses raw retrieved actions and memory-initialized proposals; and *Combined* uses both.

As shown in Fig. 4, candidate pools containing memory-derived actions generally outperform Native-only sampling in task-stage achievement. Memory-derived only often attains higher stage-wise rates, whereas on PegInsertionSide and PlugCharger, the Combined pool progresses beyond reach and contact to deeper manipulation stages such as grasp and lift. Thus, neither pool composition is uniformly superior, suggesting complementary roles for native and memory-derived candidates.

We next compare the two stronger candidate-pool variants over repeated selector adaptation on PegInsertionSide and PullCube. Fig. 5 shows that the two candidate pools exhibit distinct adaptation behavior. On PegInsertionSide, *Memory-derived only* achieves higher absolute grasp and lift rates in Rounds 2 and 3, while the *Combined* pool shows a consistent increase in these later-stage achievements across rounds. On PullCube, the difference is more pronounced. The *Combined* pool reaches substantially higher pull progress

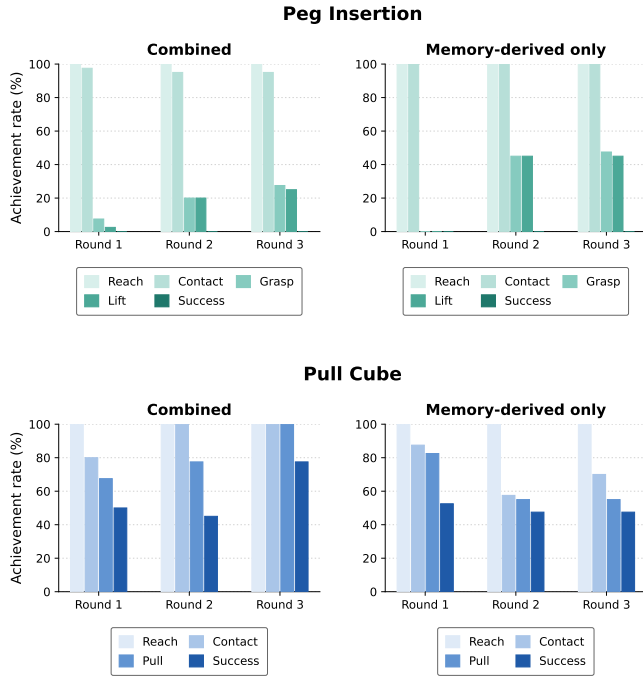


Fig. 5. Round-wise adaptation with *Combined* and *Memory-derived only* candidate pools on PegInsertionSide and PullCube.

and task success by Round 3, whereas *Memory-derived only* declines from its Round 1 performance and does not recover to the same level in later rounds. These results indicate that higher absolute performance from memory-derived candidates does not necessarily imply better behavior under repeated selector adaptation. Retaining native VLA proposals alongside memory-derived candidates may provide complementary action alternatives that become useful as the selector adapts to deployment-visited states.

D. Task-Specific Candidate Preference Criteria

For selector training, candidate rollouts from the same state are compared using task-specific lexicographic progress criteria. These criteria are used to generate candidate preferences and are distinct from the stage-wise metrics used for episode-level evaluation. The terms are compared sequentially in priority order rather than combined into a weighted scalar score. When candidates are indistinguishable under higher-priority criteria, subsequent terms provide progressively finer comparisons. In particular, lower-priority continuous measures, such as approach distance and object displacement, help reduce ties between otherwise similar candidate rollouts.

ACKNOWLEDGMENT

This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government(MSIT) (RS-2022-00143911, AI Excellence Global Innovative Leader Education Program). This research was also supported by a grant from the Institute for AI and Social Innovation at Yonsei University (Project No. 2026-22-0335).

TABLE II
LEXICOGRAPHIC CRITERIA FOR SAME-STATE
CANDIDATE PREFERENCES.

Task	Candidate preference criteria (highest → lowest priority)
PlugCharger	Success → insertion progress → socket alignment → controlled lift → stable grasp → bilateral contact → TCP-to-charger proximity
PegInsertionSide	Success → insertion progress → insertion alignment → controlled lift → grasp stability → bilateral contact → TCP-to-peg proximity
PullCube	Success → cube-to-goal progress → goal-directed cube displacement → TCP-to-cube proximity
PushCube	Success → cube-to-goal progress → goal-directed cube displacement → TCP-to-cube proximity

REFERENCES

- [1] Octo Model Team *et al.*, “Octo: An Open-Source Generalist Robot Policy,” in *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, 2024.
- [2] B. Zitkovich *et al.*, “RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control,” in *Proceedings of The 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, J. Tan, M. Toussaint, and K. Darvish, Eds., vol. 229. PMLR, 06–09 Nov 2023, pp. 2165–2183.
- [3] M. J. Kim *et al.*, “OpenVLA: An Open-Source Vision-Language-Action Model,” in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, P. Agrawal, O. Kroemer, and W. Burgard, Eds., vol. 270. PMLR, 06–09 Nov 2025, pp. 2679–2713.
- [4] K. Black *et al.*, “ $\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization,” in *Proceedings of The 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, J. Lim, S. Song, and H.-W. Park, Eds., vol. 305. PMLR, 27–30 Sep 2025, pp. 17–40.
- [5] W. Pumacay, I. Singh, J. Duan, R. Krishna, J. Thomason, and D. Fox, “The colosseum: A benchmark for evaluating generalization for robotic manipulation,” *arXiv preprint arXiv:2402.08191*, 2024.
- [6] S. Fei *et al.*, “LIBERO-Plus: A Progressive Robustness Benchmark for Visual-Language-Action Models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2026, pp. 38 574–38 583.
- [7] Z. Wang, Z. Zhou, J. Song, Y. Huang, Z. Shu, and L. Ma, “VLATest: Testing and Evaluating Vision-Language-Action Models for Robotic Manipulation,” *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 1615–1638, 2025.
- [8] B. Zhu *et al.*, “A Factory-Floor Deployment Case Study of VLA Pipelines for Industrial Packaging Task: Workflow, Failures, and Lessons,” *arXiv preprint arXiv:2605.27461*, 2026.
- [9] M. J. Kim, C. Finn, and P. Liang, “Fine-Tuning Vision-Language-Action Models: Optimizing Speed and Success,” in *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025.
- [10] F. Ferchau, D. Pommer, and C. Axenie, “On the Efficiency of LoRA Fine-Tuning for Vision-Language-Action Models in Industrial Robotic Manipulation,” *arXiv preprint arXiv:2607.10172*, 2026.
- [11] Y. Yadav, Z. Zhou, A. Wagenmaker, K. Pertsch, and S. Levine, “Robust Fine-tuning of Vision-Language-Action Robot Policies via Parameter Merging,” in *The Fourteenth International Conference on Learning Representations*, 2026.
- [12] M. Nakamoto, O. Mees, A. Kumar, and S. Levine, “Steering Your Generalists: Improving Robotic Foundation Models via Value Guidance,” in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270. PMLR, 06–09 Nov 2025, pp. 4996–5013.
- [13] Z. Li *et al.*, “Towards Deploying VLA Without Fine-Tuning: Plug-and-Play Inference-Time VLA Policy Steering via Embodied Evolutionary

- Diffusion,” *IEEE Robotics and Automation Letters*, vol. 11, no. 5, pp. 6234–6241, 2026.
- [14] M. Attarian *et al.*, “Update-Free On-Policy Steering via Verifiers,” *arXiv preprint arXiv:2603.10282*, 2026.
- [15] X. Xie *et al.*, “Look Before You Leap: Distilling Tree Search into Action Evaluation for Frozen VLA Models,” *arXiv preprint arXiv:2607.03751*, 2026.
- [16] K. Sridhar, S. Dutta, D. Jayaraman, and I. Lee, “RICL: Adding in-context adaptability to pre-trained vision-language-action models,” in *Proceedings of The 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, J. Lim, S. Song, and H.-W. Park, Eds., vol. 305. PMLR, 27–30 Sep 2025, pp. 5022–5038.
- [17] K. Sridhar, S. Dutta, D. Jayaraman, J. Weimer, and I. Lee, “Memory-Consistent Neural Networks for Imitation Learning,” in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 45 160–45 185.
- [18] Q. Pfeifer, E. Pronovost, P. Shah, K. Khetarpal, S. Srinivasa, and A. Gupta, “Difference-Aware Retrieval Policies for Imitation Learning,” in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 87 175–87 198.
- [19] J. Zhao *et al.*, “Retrieve-then-Steer: Online Success Memory for Test-Time Adaptation of Generative VLAs,” *arXiv preprint arXiv:2605.10094*, 2026.
- [20] S. Ross, G. Gordon, and D. Bagnell, “A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning,” in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, G. Gordon, D. Dunson, and M. Dudík, Eds., vol. 15. Fort Lauderdale, FL, USA: PMLR, 11–13 Apr 2011, pp. 627–635.
- [21] P. Yang *et al.*, “RARM: Confidence-Gated Progress Reward Modeling for RL in Manipulation,” *arXiv preprint arXiv:2606.22027*, 2026.
- [22] B. Liu *et al.*, “LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning,” in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, ser. NIPS ’23. Red Hook, NY, USA: Curran Associates Inc., 2023.
- [23] S. Tao *et al.*, “Demonstrating GPU Parallelized Robot Simulation and Rendering for Generalizable Embodied AI with ManiSkill3,” in *Proceedings of Robotics: Science and Systems*, Los Angeles, CA, USA, June 2025.
- [24] C. Liu *et al.*, “On-the-Fly VLA Adaptation via Test-Time Reinforcement Learning,” in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, M. Liakata, V. P. Moreira, J. Zhang, and D. Jurgens, Eds. San Diego, California, United States: Association for Computational Linguistics, Jul. 2026, pp. 40 107–40 125.
- [25] Z. Bai, C. Gao, and M. Z. Shou, “EVOLVE-VLA: Test-Time Training from Environment Feedback for Vision-Language-Action Models,” *arXiv preprint arXiv:2512.14666*, 2025.
- [26] W. Zhang *et al.*, “TTT-VLA: Test-Time Latent Prompt Optimization for Vision-Language-Action Models,” *arXiv preprint arXiv:2606.03127*, 2026.
- [27] H. Hao, S. N. Syed, J. Ichnowski, and J. Schneider, “FAR: Failure-Aware Retry for Test-Time Recovery and Continual Policy Improvement,” *arXiv preprint arXiv:2607.01111*, 2026.
- [28] H. Hu *et al.*, “ReCoVLA: VLM-Guided Reward Compilation for Failure Recovery in Vision-Language-Action Policies,” *arXiv preprint arXiv:2606.09630*, 2026.
- [29] S. Sudhakar, J. Liang, S. Rammohan, P. Tokmakov, R. Zemel, and C. Vondrick, “Robot Critics that Sweat the Small Stuff,” *arXiv preprint arXiv:2606.21572*, 2026.
- [30] J. Kwok *et al.*, “Scaling Verification Can Be More Effective than Scaling Policy Learning for Vision-Language-Action Alignment,” *arXiv preprint arXiv:2602.12281*, 2026.
- [31] J. Zhang *et al.*, “What Frozen VLAs Already Know About Success: A Probing Study of Value-Like Structure in Foundation Robot Policies,” *arXiv preprint arXiv:2605.28527*, 2026.
- [32] S. Shin *et al.*, “Back to the Familiar Future: Failure Recovery for VLA Policies via Pre-Imagined Milestone Selection,” *arXiv preprint arXiv:2606.09258*, 2026.
- [33] J. Zhang *et al.*, “ReWiND: Language-Guided Rewards Teach Robot Policies without New Demonstrations,” in *9th Annual Conference on Robot Learning*, 2025.
- [34] Y. Mao *et al.*, “ARM: Advantage Reward Modeling for Long-Horizon Manipulation,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, June 2026, pp. 4468–4477.
- [35] A. Liang *et al.*, “Robometer: Scaling General-Purpose Robotic Reward Models via Trajectory Comparisons,” in *Robotics: Science and Systems 2026*, 2026.
- [36] N. Team and F. T. Team, “N₀-VTLA: Scaling Vision-Tactile-Language-Action Model with Latent Tactile Tokens,” *arXiv preprint arXiv:2607.23782*, 2026.
- [37] I. Xu *et al.*, “Learning More from Less: Reinforcement Learning from Hindsight,” *arXiv preprint arXiv:2607.09042*, 2026.
- [38] R. Li *et al.*, “MAP-VLA: Memory-Augmented Prompting for Vision-Language-Action Model in Robotic Manipulation,” *arXiv preprint arXiv:2511.09516*, 2025.
- [39] S. N. Syed, Y. Ahuja, A. Jakobsson, and J. Ichnowski, “ExpReS-VLA: Specializing Vision-Language-Action Models Through Experience Replay and Retrieval,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2026.
- [40] M. Oquab *et al.*, “DINOv2: Learning Robust Visual Features without Supervision,” *Transactions on Machine Learning Research*, 2024.
- [41] C. Bianchi *et al.*, “Robotic Policy Adaptation via Weight-Space Meta-Learning,” *arXiv preprint arXiv:2606.07217*, 2026.